

ממון 14

פרויקט גמר

שם: צור פינטו לזר

README

תוכן עניינים

איך להשתמש באסמבלר

מבנה הפרויקט

איך כל שלב עובד

הנחות יסוד ובחירות שנעשו

נספחים

איך להשתמש באסמבלר?

להיכנס לספרייה של הפרויקט ולהריץ

make

./assembler file1.as file2.as file3.as

לדוגמה:

```
tzurpl@tpl ~/U/m/project (main)> ./assembler input_output/input1.as
Processing input_output/input1.as...
Successfully assembled input_output/input1.as
tzurpl@tpl ~/U/m/project (main)> █
```

מבנה הפרויקט

```
tzurpl@tpl ~/U/m/project (mai
├── assembler
├── assembler.c
├── assembler.o
├── errors.c
├── errors.h
├── errors.o
├── first_pass.c
├── first_pass.h
├── first_pass.o
├── globals.h
├── input_output
│   ├── input10.as
│   ├── input1.as
│   ├── input2.as
│   ├── input3.as
│   ├── input4.as
│   ├── input5.as
│   ├── input6.as
│   ├── input7.as
│   ├── input8.as
│   ├── input9.as
│   ├── input_output.png
│   ├── output1
│   │   ├── input1.am
│   │   ├── input1.ob
│   │   └── out1.png
│   ├── output10
│   │   ├── input10.am
│   │   └── out10.png
│   ├── output2
│   │   ├── input2.am
│   │   ├── input2.ent
│   │   ├── input2.ob
│   │   └── out 2.png
│   ├── output3
│   │   ├── input3.am
│   │   ├── input3.ent
│   │   ├── input3.ext
│   │   ├── input3.ob
│   │   └── out3.png
│   ├── output4
│   │   ├── input4.am
│   │   ├── input4.ent
│   │   ├── input4.ob
│   │   └── out4.png
│   ├── output5
│   │   ├── input5.am
│   │   ├── input5.ent
│   │   ├── input5.ext
│   │   ├── input5.ob
│   │   └── out5.png
│   ├── output6
│   │   └── out6.png
│   ├── output7
│   │   ├── input7.am
│   │   └── out7.png
│   ├── output8
│   │   ├── input8.am
│   │   └── out8.png
│   └── output9
│       ├── input9.am
│       └── out9.png
├── Makefile
├── preproc.c
├── preproc.h
├── preproc.o
├── README.md
├── second_pass.c
├── second_pass.h
├── second_pass.o
├── table.c
├── table.h
├── table.o
├── util.c
├── util.h
└── util.o
```

איך כל שלב

עובד

preprocess

בפשטות השלב הזה לוקח את הקובץ אסמבלי, שומר את
תוכן המקרואים ברשימה מקושרת של מקרואים ואז
מרחיב אותם היכן שהם מוזכרים

השלב הזה בודק מספר שגיאות שיכולות לקרות:

1. אם שורה היא ארוכה מידי

2. תקינות שם המקרו

3. אם הגדירו מקרו פעמיים

4. אם יש אקסטרה טקסט אחרי מתי שהשורה

מסתיימת

first pass

השלב הזה אחראי על לעבור על הקובץ אסמבלי הפרוס, שומר את הלייבלים והדירקטיב וההוראות לטבלאות שלהם, אחרי זה הוא מעבד את כל ההוראות לפי סוגן וממיר אותם לקוד מכונה הנשמר ברשימה מקושרת אחת לקוד והשנייה לדאטה, ההמרה נעשת על ידי unions וbytestructs שעוזרים בפיצול כל הוראה לבתים, ובנוסף הוא מטפל בכתובות ובהזזתן, ולבסוף שומר את כל השגיאות בקוד שמצא.

second pass

השלב הזה אחראי לעבד את כל השורות שהמעבר הראשון לא הצליח, ולפתור את החוסרים במידע של לייבלים שהוזכרו לפני שנמצאה הכתובת שלהם, הוא מעבד את שורות ה `extern` ו `entry`, העיבוד של השורות נעשה באותה הדרך כמו במעבר הראשון עם `unions` ו `bitfields` לקוד מכונה שנשמר ברשימות המקושרות שהוזכרו במעבר הראשון, ולבסוף שומר את כל השגיאות שנמצאו.

assembler

השלב המרכזי בפרויקט, בשלב הזה האסמבלר קורא למעברים, ורק אם הם חזרו ללא שגיאות הוא לוקח את כל הרשימות עם הקוד מכונה ומדפיס את השורות לקובץ קוד מכונה יחד עם הכתובות ועם ה IC ו DC למעלה, ואם לפחות אחד מהמעברים או ה preprocessor יוצאים עם שגיאות אז הקובץ לא נוצר והשגיאות מודפסות. לבסוף הוא משחרר את כל הרשימות והטבלאות.

error detection

בפרויקט בשביל לנהל את כל הזיהוי שגיאות וסיווגן יצרתי קובץ שהמטרה שלו היא לסווג השגיאות לסוגים עי שימוש בenum עם כל סוגי השגיאות, ופונקציה האחראית על הדפסת השגיאה לפי סוגה, לכל סוג יש פרמטרים משלו.

בנוסף לכך יש רשימה מקושרת האחראית להחזקה של כל השגיאות שזוהו במעברים הראשון והשני, ופונקציה מתאימה לה שהתפקיד שלה להדפיס את כל השגיאות אחת אחרי השנייה ולמיון את השגיאות כדי שההדפסה תהיה לפי סדר השורות.

Utility

בפרויקט במקום שדברים שהם בשימוש בכמה פונקציות
ובכמה קבצים במקום לכתוב אותם מחדש כל פעם אז
יצרתי קובץ עם פונקציות עזר לפרויקט, כמו איסוף
ארגומנטים וטוקנים, עיבוד חלקי הוראות, ניהול טבלת
ההוראות, בדיקה של שמות של לייבלים ומאקרואים,
כלים לניתוח מחרוזות וכו

הנחות יסוד

ובחירות

שנעשו

הנחות כלליות

- בשביל כל המערכים והטבלאות השתמשתי ברשימות מקושרות
- כל מספר שאינו 0 או 1 הוגדר עם שם, חלק בקובץ של הגדרות גלובליות
- כאשר פונקציה מקבלת ערך שהיא לא אמורה לשנות היא מקבלת אותו כ `const`
- אין שימוש בביטויים מסוג `static`, אפילו לא פונקציות (לא היה ברור מהפורם מה מותר אז הלכתי על בטוח)
- אין מערכים גלובליים (אמרו שאסור בפורם)
- בקובץ קוד מכונה שנוצר ההדר שם (IC DC) מקודם בטאב כדי להיות באמצע (לא היה ברור מהדוגמה בחוברת בקורס אם צריך או לא ואם כן כמה)
- השורות מנותחות על ידי פיצול לפי טוקנים (ארגומנטים) שנשמרים ואז משוחררים כשמסיימים להשתמש בהם
- כי הפונקציה `strdup` לא נכללת בספריה שאמרו שמותר להשתמש בה אז בקובץ `util` יצרתי אותה

- במקום להשתמש רק ב malloc ו realloc הרגילים יצרתי פונקציות שברגע שהקצאת זכרון נכשלת התוכנית יוצאת ככה שבמקום לעשות if בכל פעם של אלוקציה אז זה יבדוק לבד
- הטבלת ההוראות היא היחידה שהיא לא רשימה מקושרת ופשוט כתובה אבל היא בתוך פונקציה שנותנת גישה למערך כי אסור מערך גלובלי [לפי הפורם]
- גם אם יש תקלה בהגדרה של מאקרו הפרה מעבד עדיין שומר את המאקרו כדי שהוא יוכל להמשיך לעבור על הקובץ ואם יש עוד שגיאות למצוא אותן
- כידוע האורך של השורה צריך להיות 80, ומהחוברת ומהפורם לא הבנתי אם זה כולל התווים הסופיים \ ו \0 או לא אז אני בודק בפרויקט ל 82 תווים
- לא השתמשתי ב break ו continue ו switch cases כי לא הבנתי אם מותר או אסור בקורס [ממה שהבנתי מההרצאות אסור]
- הנחתי שאסור תווים לבנים לפני הערה בקוד [לפני ;] (לא ממש היה לזה הסבר אם מותר או אם זה נחשב טקסט לפני הערה)

הנחות לפי שלב

preprocess:

- גם אם יש תקלה בהגדרה של מאקרו הפרה מעבד עדיין שומר את המאקרו כדי שהוא יוכל להמשיך לעבור על הקובץ ואם יש עוד שגיאות למצוא אותן
- הפרה מעבד בונה את הקובץ אסמבלי המורחב בתוך מחרוזת בזכרון ורק אחרי שהוא סיים לעבור על כל הקובץ אם אין שגיאות כלל רק אז הוא יוצר קובץ am

first pass:

- השורות של הקובץ אסמבלי המורחב מתורגמות לקוד מכונה לא על ידי הזזות של ביטים בעת החישוב אלא על ידי struct-ים לכל סוג הוראה ששם יש bitfields המפצלים כל הוראה לאיך שהיא אמורה להראות בקוד מכונה, וגם struct לפיצול הקוד מכונה ל4 בייתים למען הדפסתם בהקסה בסוף התהליך, אותו סטראקט משמש גם להמרה של הדירקטיבים של הדאטה לבתים.

second pass:

- המעבר השני תמיד רץ, בלי קשר למעבר הראשון ככה שהשגיאות שנמצאות בקובץ אינפוט כולן ימצאו.
- המעבר מתרגם את השורות שלא תורגמו לקוד מכונה במעבר הראשון באותה צורה כמו המעבר הראשון עם סטראטקטים הכוונה
- ההנחיה. entry מעובדת רק במעבר השני, כי קובץ האנטרי נוצר רק כאשר לפחות סמל אחד של אנטרי נמצא, וגם ההחניה extern מעובדת רק במעבר השני מאותה סיבה כדי למנוע יצירה של קבצים ריקים

error detection:

- בפונקציה האחראית על סיווג סוג השגיאה הסיווג לא מבוצע בswitch מאחר ולא הייתי בטוח אם מותר להשתמש בו ובbreak וcontinue ולכן נמנעתי